So welcome to the beginning of this
tutorial, and I'm gonna show you what
we're gonna build today, so that you get
a flavor of what you're gonna be able
to get to at the end of this tutorial.
here we go.
By the end of this tutorial,
you will have what I call an
endless survival beat-em-up game.
You play as this pirate, and then you
will have continuous waves of skeleton
enemies that will come, and it will
get increasingly more difficult as more
and more of them turn up over time.
and while this game may seem quite simple,
the key thing here is to enable you to
understand the fundamentals, such as
loading in game art, how to implement
collision detection, how to implement
movement, everything that you learn today
will be applicable regardless of what
games you're trying to build whether
it's a top-down game, whether it's an
isometric game, whether it's an arcade
game, because all of the concepts that
you learn today will be applicable.
And you will be building this game from
scratch, from this starting folder of
files, and I will take you prompt by
prompt all the way from start to finish.
And the goal here is to help you
build the muscle memory to be
able to build games such as this.
All right, before we jump in, I've
made all of the files that you need
for free as part of my Insiders Club.
link is in the description below, and
you'll have everything you need to
follow along with today's tutorial.
and if you would like to dive
deeper with even more resources
on AI-powered game development,
You can check out vibegamedev.com.
Within it, you'll find the full

written and broken down tutorial of
what we're going to go through today.
it also includes a lot of game
development specific agent skills.
and the full source code of all of
my other projects that I've been
working on throughout this channel.
there's also a private exclusive Discord
community where fellow game development
Vibe coders hang out sharing tips
and tricks with one another to help
each other along on their journey.
, so once you've got your project onto
your computer, it's time to get going.
I'm just gonna walk you through
what's inside the folder so that you
know where we are starting off from.
A lot of these files that you see here
are just called boilerplate, it's just
the basic things that are required for the
game to get up and running so that you can
play it on your computer in your browser.
There are several things that I
would like to call your attention
to, and let's just start off with
what's inside this public folder.
Now, the public folder is where
we will store all of our game
assets, and you will see that
there are already two folders here.
One is called the pirate and one is
called skeleton, and within it, it
already contains all of the assets
that you'll be using within your game.
in the animations folder, you have the
attack, death, hurt, idle, jump and walk.
And within these are what
you call sprite sheets.
a sprite sheet is a combination of all
the individual frames such that your
character can animate within the game.
So if you look at this death
sprite sheet, if you play this in
sequence, you will get an animation.

you can just look at the preview .gif
file over here, when it's stitched
together, you get a complete animation.
the folders that you have here has
all the sprite sheets for the skeleton
as well as the pirate character.
So you already have everything
that you need to go ahead and build
this game that you see over here.
So the other thing to call attention
to is that you will see that you
have a .agents and a .claude folder.
these two folders contain things called
skills, and you will see that the
skills that are present already are the
Phaser and Phaser 4 game dev skills.
so in today's tutorial, we'll be
building this 2D game using the Phaser
game engine, and that's the reason why
we have loaded in these Phaser skills
It's basically an instruction set as
well as a knowledge base that gives
your AI superpowers in a related domain.
So in this case, it's going to
be an expert in building games
using the Phaser game engine.
because the skills are provided for
both Claude Code and any other AI
agents, you can use Cursor, Claude Code,
Open Code, Zencode, or the Codex app.
I highly recommend the Codex app
because the Codex app does provide
a lot of additional functionality as
part of your ChatGPT subscription.
for today's tutorial, I'll be
using the Codex app from OpenAI.
in the Codex app, we're gonna
go to File, I save this as VGD
Pirate Survival Beat Em Up, so I'm
gonna open this right over here.
and once you've loaded it up,
you will see that this is the
folder that we're gonna be in.
You want to be working locally, and you

want to be in the start branch, okay?
All right.
And in this case, if you know what you're
doing, you can go to full access, which
means that Codex will not prompt you for
any permissions, and it should be fairly
safe when you're doing this tutorial.
if you want to be more cautious, you can
do auto-review, which means that Codex
will only ask you when it thinks that
it's doing something more dangerous.
At the time of this tutorial, I would
suggest you use five point five, and
you can switch between medium or high.
It If you have a higher tier of
the Codex or ChatGPT subscription,
like Pro, you can go to fast, which
will make things a lot quicker.
If you have a lower subscription
and you can just use medium.
but if you are on a higher
tier subscription, you can
go ahead with using high.
So as you can see, in this case, I'll be
using high and fast but feel free to stay
on the standard speed in order to proceed.
Okay?
so the first thing that we're
gonna do is run this project.
this will look like a magic command
if you've never used it before.
but let's just go here and do it.
if you go over here and click
Toggle Terminal, or you can do
Command J, this will bring up this
little console below over here.
Don't worry about it.
All you wanna do is you want
to do NPM install, okay?
this will just install all of the
dependencies to run your game.
The next thing you want to do
is to run NPM run dev, okay?
And You will see something

like this pop up.
It will say it's ready, and then you will
have something called Local and Network.
What you wanna do now is just
open this up in your browser.
You can hold down the Command button
or the Control if you're on Windows,
and then click on this button, and
This will now open up the game within
this window that you see over here.
Now, the problem with this is that
while it's great that it shows up over
here, It will appear quite small, okay?
Unless you click on this
Play button over here.
And then now once you're within
it, you can do up and down.
You can go to Settings.
you can mute it, and then you
can go back to the main menu.
And then the other thing that
you can do is that you can go to
Sandbox, and You will see that
this will be what we start with.
It's just a very simple square that's
walking around, that allows you to use the
arrow keys to walk around on the screen.
by the end of today's tutorial, you
will get from this very simple-looking
sandbox, which is just a starting point,
to having this fully-fledged complete game
that you see over here, and that we were
discussing at the beginning of the video.
Now one thing that I encourage you to
do is because this screen is embedded
within your window, and if you're working
on a laptop and a small screen like me,
what you can do is that instead of just
clicking on this here, you can just copy
this location, and then just copy and
paste this location onto your browser.
with that, you have a much larger
screen version of what you see
over here, rather than having this

collapsed within this window over here.
And the reason why you want to do
this is that when you're in this
full screen debug mode, you have this
console on the right, which will help
you to perform troubleshooting steps
in the event that you hit any issues.
So for example, As I'm moving around
with this character that you see over
here, you will see that the input on
the right, as well as the location of
the pointer, is updating in real-time.
This is part of today's tutorial.
You will see that having debug
visualization such as this will
help you to troubleshoot if you
encounter any errors along the way.
all right.
once you are able to get to this
stage, you are all set because the
game is running, your project is
loaded up in Codex, and you have all
the files, we discussed over here.
So you will be ready to go ahead
and begin implementing this game.
all right, so let's get started, So
we're gonna be in the Codex app and
you want to make sure that you've
opened up your project folder.
in this case, it's VGD Pirate Survival
Beat 'Em Up Game, Beat 'Em Up.
double-check that this is the case,
and then you wanna be working locally
If you cloned this from GitHub,
you should be on the start branch.
Don't worry too much about this because
we will not be doing any version
control as part of today's tutorial.
if you know about Git, you will
know that this is the branch that
is the beginning of the project.
the first thing that we're gonna do
is, if you recall, we have this nice
background art that you see over here,

and I thought that the first thing that
we would like to do is to show you how
I actually got hold of this background.
And this is the prompt
that we're gonna use today.
So I'm gonna look through this.
I'm gonna take you
through this step by step.
All right?
So the first thing it's gonna do
is that I'm gonna ask it to look at
all the images that we already have.
And if you recall, we have two folders
of animation sprite sheets, including
ones for the pirate across all of
the different states that we have.
And then we also have one more
for the skeleton character.
So what we're saying is that study
these assets because this is the style
of the game that I'm trying to create.
I want it to create a single level
with no scrolling using these two
characters, and I want it to generate
four candidates, make sure that there
is sufficient space to walk around both
horizontally and vertically, and there
needs to be a limit to the vertical range
because this is gonna be a backdrop.
Now, one thing I'm gonna do here is
that you can see that I'm saying I
want to generate these four candidates
using image gen. if I click over here,
I'm gonna use the image generation
skill that is built into Codex and that
piggybacks off your ChatGPT subscription.
So this allows you to generate
images for your game without
needing any additional subscription.
Finally, I'm gonna say place these
into public assets backgrounds
and use parallel sub-agents.
All right, so now I'm gonna kick
this off, and I'm gonna explain

to you what I'm trying to do.
in the original tutorial related to
this project, I created four different
backgrounds before I landed up on
the final version that you see over
here, which is like on the ship.
I asked the AI to generate four different
backgrounds, and I got a pirate cove,
one that's on the docks, one that's on
the ship, and one that was in the ruins.
the reason for this, is that you want
to have some room to experiment and to
see what suits the style that you want.
if you look over here, Codex will
look at the two images that you
have within your project, okay?
And it's able to look at these images,
it's going to say, "Okay, I now understand
what the style of game that you're going
for," which is really useful, okay?
if you remember, I said
use parallel sub-agents.
you will have seen that these four
background agents have spawned, What's
happening here is that you want four
different backgrounds, rather than doing
it sequentially, you're telling Codex to
spawn it in parallel so that you get all
four backgrounds at around the same time.
this is a very powerful tool that
you should be making use of when
you're using something like Codex.
all it takes is just a single line like
this, and it automatically will spawn
all the four parallel threads for you.
as you can see here, these will now start
to land and appear within the artifacts.
this is the reason why I, encourage
people to use the Codex app as part
of this tutorial because not only does
it work off your ChatGPT subscription,
which you probably already have,
but the overall user interface has
gone through a lot of changes that--

made vibe coding a lot easier.
All right.
we are landing with these four
candidates, and so Codex should
be wrapping up pretty soon.
Because we are using the inbuilt
image generation, Codex will
actually store these images in a
different part of your computer.
So what it's gonna do is do this
final step that we set over here,
which is put it into Public Assets
Backgrounds, it's already created
a folder, and it's going to put the
four backgrounds into this location.
And yep, just as I said
it, it's landed here.
the reason why you want it to be
put over here is because our game
needs to be able to be self-contained
and have access to all of these
backgrounds so that we can load it in.
And here he is, and it's completely done.
It took about seven minutes to get all
of these images, and they are all great
candidates for us to put into the game.
so now that we have these four images,
the next step is teach our game and
the AI to load in the backgrounds as
well as the two different characters.
So the way that we go about teaching
our AI as well as our game to
know about these new assets is
to-- We're gonna do the following.
say, "Look at the assets that exist,"
and then an assets index.json that
will serve as a canonical index to all
available sprites in the game." this
means take into account single sprites
like the backgrounds, as well as the
sprite sheets used for the animations.
specifically for the sprite sheets,
we need frame size, number of frames,
offsets, and any other metadata useful

for integration into a Phaser game.
So I'm gonna kick this off, and
explain what we're trying to do.
But effectively, even though we have all
our assets with the different backgrounds
that we have in the game, our current
game still doesn't know about them, right?
Right now, all it has is this blue
square moving around in a black, screen,
and it needs to load in these assets.
What we're trying to do here is create
something called an index or an asset
index, and the goal here is to create
a single file that will point to all of
the different assets that are available
for us to integrate into this game.
And the reason why you want to have a
single file is so that any time that
the AI needs to look up a particular
sprite sheet or character or background,
it doesn't need to list all the
files within the folder anymore.
All it needs to do is just check
this single file, and it will
be able to find it immediately.
this will help save on your usage
and your tokens The second reason for
having a single file serving as an
index is that if you are adding in new
animations or backgrounds or sprites,
you can tell your AI to update the
index file, and the game will know
where and when to pick things up.
this is good practice.
If you watch any of my other videos, you
will know that this is something that I do
regardless of any game that I'm building,
whether it's a 2D game, whether it's a 3D
game, I always have an index JSON file.
And so just jumping back here, it
says it has created the index.json,
click it over here, and you
will be able to see the result.
Now, let me just walk you through

what's within it over here.
So if you look, the index.json now has a
couple of really important things to note.
So it tells you when it was generated,
where the assets are stored, and most
importantly, now it has listed that it has
four backgrounds and a lot of information
related to what these backgrounds are.
And then the other thing that it
has is that it has every single
animation that you see over here.
And the best way to show this
to you is to actually take up
one, such as the hurt animation.
Okay.
I'm just gonna pop this to the side and
then keep the index.json on the left.
So now if we look for hurt, you will see
here it's telling you that this entire
sheet is twelve eighty by five twelve.
There are five columns and two rows.
There are six frames, so one,
two, three, four, five, six.
and then it has information on the frame
rate that you want to run, as well as the
different anchor points for each of these.
The anchor points are really important
for it to be able to play animations
such that they're not all jumping around.
you don't really have to go into
too much detail about what this is.
This is really meant for the AI to be
able to understand what assets you have.
Now that we've got all of our assets
and we've got this index.json, it is
time to move on to the fun part where
we actually start to integrate all
of these assets into a playable game.
So
I'm gonna use this prompt that you
see over here, and there are two
skills that we're gonna make use of,
included in this project folder Now,
Phaser Game Dev is because we are

using the Phaser game engine, so this
contains a lot of information and
knowledge about how to utilize it.
And Phaser Phor is because it's the latest
version of Phaser, and there's a couple
of new things that we want to make use
of so that the game runs really well.
And so you can see here, we're gonna
integrate the pirate into the game.
We will support W, S, D,
and arrow keys for movement.
We will have Z for attack, and
it should animate correctly.
And over here, what we're gonna do is
that we're gonna say we want to use
background two as our chosen background.
All right, and then we're
just gonna kick this off.
and you can see from my prompt, it
doesn't take a lot at this point
because it has already enough
information about what it needs to do.
going to be able to just take this very
simple prompt and give you something that
resembles a working game very quickly.
All right.
So you can see here it's gonna
use both the Phaser skills, right?
Phaser 3 is driving the
implementation, and Phaser 4 will
be used to standardize it and make
use of some of the newer features.
if you want to look at, different
skills that you have, you can go
into Plugins, and then onto Skills.
You will see that Codex comes with
several inbuilt skills, like the Image
Gen, which is the one that we were using.
and because I have included additional
skills as part of this project,
you have the Phaser Game Dev,
As well as the Phaser Phor one.
All right, let's jump
back into our game here.

And you can see that it's looking
at what we have in the game already.
And if you recall, the game currently
has just a single screen with
the blue character moving around.
This is called the sandbox scene.
So the AI is now thinking,
what should we do now?
Should we create a new menu screen, or
should we just replace what's already
there?" And it's decided that it's just
gonna go ahead and replace what's already
there If you wanted it to create a new
menu item, because of the boilerplate
that we already have, you can simply say,
"I want a new screen," and it will do
that and retain this sandbox over here.
And there you have it.
it's already started to integrate
the character into the game.
And let's just play around with this.
WASD works, all right?
So I can move up, left, down, and
I can use the arrow keys as well.
If I press Z, it does the attack.
And this is looking great, right?
Like already, you have
something that resembles a game.
It's done a really good job of making
sure that it doesn't step into, the water.
But you might notice that if it walks
too far to the left, it might end up
walking on some of the props over here.
Okay?
And I'll just show you the reason why.
you can see that there's this
wall bounds, that constrains
where the character can walk.
And you can see over here it's correct,
but if we walk further to the left,
you will notice that it's a little
bit too far and we should be making
this wall bounds a little bit shorter.
But more on that later.

Now, if you're working in Codex, You
will probably notice that sometimes it
will launch a separate browser for you.
And this is one of the cool things about
using the Codex app, is that it has
something called browser use built in.
as you can see here, it's going to
resize the browser, and then it's going
to look at the files, And then it's
going to try to take several snapshots
of the game as it loads up the browser.
the idea here is that it's trying to
do an automated verification about
whether its changes made sense.
in some cases, you will need to
say allow over here so that it's
allowed to press keys for you.
And you can see it automatically
pressed the sandbox button so
that it could load up the game.
and this, is a really powerful tool.
And you can see it's looking at
what's on the page right now.
I'm gonna shift this side by side
so you can see what's happening.
effectively, what it's done is that it
took a snapshot, and then it pressed a
browser key, and then now it's trying
to take another screenshot again.
Okay?
So what I'm gonna do is that I'm
gonna steer this a little bit.
I'm gonna say, "Do not clear or delete
any Playwright-" screenshots, so that
I can keep track of our progress.
Okay, and I'm just gonna press Enter
here, and you can see this called Steer.
Okay, and this is a nice
thing about Codex as well.
You can either queue this message to
happen at the end, or you can actually
start to steer it and as you can see
here, I managed to, steer it so that
it doesn't delete the screenshots

because I want to keep track of this
as we go on through this tutorial.
I'll leave every Playwright screenshot
in place because I managed to
steer it, and so it didn't lose
context of where it needed to be.
and what I'm gonna do now is
Allow for this chat so that
it doesn't keep prompting me.
Another thing that you can do is set this
to auto review so that it will not keep
prompting you whenever it needs approval
for something like this, and only when it
looks like it's doing something dangerous.
if you jump into the game, and allow
it to continue its testing over
here, you will see that everything
is working as we expected it to do.
So we are able to walk around, we
are able to attack, and This is
already starting to take shape.
So what we're gonna do next is that we're
gonna start to make this game a lot more
interesting by integrating the enemy and
have a bit of challenge into the game.
unfortunately, I recorded this bit and I
accidentally, killed the screen recording.
So I lost, the raw footage, but I'm just
gonna give you an overview of what I did.
the next thing that we want to
do is to make this game more
interesting by adding the skeleton
character into the game as an enemy.
now let's add a skeleton enemy
using the skeleton sprite.
The enemy is slower than our character,
but can attack our character.
When he attacks, the hurt
animation needs to play.
There is a knockback of the player
in the direction of the hit, and the
enemy needs to pause a while before
continuing to chase the player.
And let me explain to you

what is going on here.
But, so if you see this final version
of the game, when the enemy attacks
You will see that the player gets
hit backwards away from the enemy.
And the enemy also doesn't
give chase straight away.
There's a bit of a pause.
Now, what I'm trying to demonstrate
to you here is that while vibe coding
can get you very far, sometimes you
find that it's quite difficult to
explain the kind of effect that you're
trying to get out for your game.
using terminology such as this, even
things like sprite sheets and sprites,
And in this case, using something like
knockback and also having some sense
of how the game feel should be like,
which is having this enemy pausing.
These are things that are more important
than understanding how to implement it
because AI will handle that part for you.
So the main thing when you're vibe
coding games is that you want to be in
a position where you can describe it
in the best possible way for the AI to
be able to go ahead and implement it.
let me just show you, what the end
result was at the end of this stage.
so if we jump to the game now,
you will see that the skeleton
has been integrated correctly.
The skeleton can hit the player,
and there is the knockback as well.
and you will see that, There is
always a brief pause before the
skeleton starts chasing again.
in the past, if you wanted to
implement something like this, it
would have taken you quite a while.
This just took a single prompt as long
as you know the right kind of terms to
use to get the effect that you want.

Now, you will see that there is
a problem with this attack while
everything looks fine at first glance,
There are several things
that you will notice.
So the first problem that you'll
notice is that even before the skeleton
hits me, I'm really suffering from a
damage, and that doesn't look right.
Now, the other thing is that I don't even
need to be in the range of the weapon,
but the player is also getting hit.
These are bugs that completely break
the immersion and fun of your game,
and that's why it's important to learn
how to troubleshoot this using debug.
so what I'm gonna do right now is I'm
gonna show you the reason why this
is happening in this final game, I'm
gonna pause the game for a while,
So you can see on the visual bounds.
So this is how the characters are
placed within the world based on their
two fifty-six by two fifty-six frames.
But the main thing that you want to see
are the hitboxes and the attack boxes.
So the hitboxes just tells you
where exactly the weapon needs to
hit in order for it to register.
And you can see it's not the entire frame.
It's just the box And this applies both
for the skeleton as well as for the player
the other thing to highlight is that when
we are walking, there are no attack boxes.
It's only when the skeleton starts
swinging that you see the yellow attack
box and only when I start attacking
that my orange attack box comes out.
this is what makes the
game feel a lot better.
Because when all of these are in
place correctly, It only registers
the hit at the right frame, and this
makes sure that the game doesn't

feel broken as we see over here.
Okay?
So what we're gonna do now is add
the debug drawing so that we can
start to visualize what's going so
for us to be able to visualize this,
I'm gonna use this prompt over here.
I'm gonna say, "It is worth being able
to visualize the visual bounds hitbox
and the attack boxes, which should
be separate elements for each actor."
The pirate and the skeleton.
Add these debug bound toggles
to the right-hand panel.
So we're gonna kick this off.
And so what we have here, you can see
that the bounds are now showing over here.
And if we go into the game, I'm
gonna turn on the bounds so that
we can actually visualize this.
So immediately you can
see several problems.
So you can see immediately that the
hitboxes for the character as well
as the skeleton are not accurate.
And secondly, the attack boxes
of the character and the skeleton
are way too big and are already
appearing on the first frame.
So if I resume this, watch this.
the moment the skeleton swings back the
weapon, it already registers as a hit.
and that's not the feature that we want.
Okay?
So hopefully with these changes, you
can understand the value of being able
to see these debug, bounds in the game.
And as you can see here, Codex
has taken a screenshot of the game
just to make sure that the debug
bounds are showing correctly.
Okay?
So now that we know what are the
problems, let's try to fix this.

So we're gonna fix the first problem where
the weapon is showing up all the time,
So this is the prompt that I'm gonna use.
I'm gonna say, "Looking at the
sprite sheets, the extent of the
weapon only happens in frame four
for the pirate and the skeleton." So
you can see one, two, three, four.
Only at the fourth frame does
the weapon actually stick out.
everything else is just
like part of the winding up.
and this is the same thing
for the skeleton as well.
And you can see over here,
yeah, only in the fourth frame.
Okay?
so we should reflect this, otherwise
the player gets hit even in the wind up.
Take this opportunity to do the same for
the pirate and implement damaging the
skeleton in a similar way with kickback.
So if you recall, we actually
have not implemented attack, so
it's very one-sided right now.
So the player can't attack
the skeleton at all.
So we're gonna replicate the
functionality that we see with the
skeleton attacking the pirate to be
able to work the other way as well.
So we're just gonna kick this off
And the goal here is so that we'll be
able to have a much better hit detection
and also the ability for us to hit
the skeleton as well let's take a look
at where it's gotten to right now.
I'm gonna pause the game for a
while and take a look at all these.
you can see that we have all of the attack
boxes now, and this time, only when it's
in frame four do you see that the yellow
and orange attack boxes get highlighted,
which means that only when that frame

is playing does the hit register,
and this is looking a lot better now.
So when there's a wind-up, I
still have a chance to run away,
but the problem still stands.
so in terms of the timing of
the frames, it still works.
but the hitboxes are still not
right, and the attack boxes are
still not right in terms of size.
So now let's fix the sizing of these
hitboxes so that we we at least have
a chance to run away from the enemy.
otherwise it makes it a lot difficult.
So just to show you the power of,
the browser use over here, it's taken
a screenshot just as the attack is
coming up, just to show that it's
indeed only highlighted at the
correct frame, and that's how it
verifies that it's working correctly.
So what I'm gonna do now is
I'm gonna fix the bounds issue.
So with the bounds, I noticed
that both the attack bounds
are low and large in height.
Given the weapon's thinness, this should
not be so big in height, and it should be
positioned higher relative to the bodies.
All right, so now let's kick this off
so that we can end up with a situation
where this gets fixed correctly.
So let me just show you the hitboxes here.
So you can see the attack
boxes are a lot smaller.
they are positioned correctly relative to
the, character as well as the skeleton,
So this gives you a chance to evade the
attack rather than what we see over here,
where the hitbox is just way too big Okay.
All right, so you can see it says
here that it's gonna tighten the
attack boxes to match the, stick
shape better, and then we'll see

whether or not this gets it right.
Sometimes it might get things
wrong, so let's just take a look
at how things are right now.
So let's just turn it up,
and I'm gonna pause it.
you can see while it's gotten the thinness
correct, it's still a little bit too high.
So we can pause it, and take a screenshot.
Okay?
and then I wonder whether it's going to
be able to figure things out on its own.
But while it's running this, we can
jump-start and paste the screenshot
here and say, the thinness of the
attack is correct, but they are still
too high relative to the body of
the characters." and you can press
Escape to interrupt the current
processing rather than steering it.
You saw the Steer button just
now, which you can use to let it
continue what it's doing and then
gently steer it towards an outcome.
But in this case, I want to get
it to fix this correctly now so
that it doesn't waste more time.
So the attack boxes are a lot lower now.
Okay.
And this is actually
a good, size, I think.
So you see, I can now evade the attack.
And you can spend more time,
to tweak these boundaries.
So what you wanna do generally is,
make the, collision bounds for your
character a little bit smaller than
the enemy, so that it's easier for
the player to hit the enemy and harder
for the enemy to hit the player.
Same thing with the attack bounds.
You can make it longer so that the
player is likely to hit the, enemy more
than the enemy is to hit the player.

you can continue tweaking
this as much as you want.
for now, let's move on to the next
stage where we're gonna start to
have a health system So this is not
just gonna be an infinite running
game where you just slap each other.
so what we're gonna do
now is gonna use this.
So we say, let's make a health
system, give them to both
the player and the enemies.
The enemy's health, appears as a health
bar, and the player's health appears in
the top left-hand corner in a longer bar.
And we want this to change color
as it depletes, so green, amber,
red, and it blinks when it's red.
And the player's health is five,
the skeleton health is three.
Hit damage is one always.
Okay?
So we're just gonna kick this off, and the
idea is that you will see something like
what we see over here, where the health
bar is, above the enemy and the player's
health is at the top left-hand corner,
so it's more evident to the player.
And you can see every
hit is about one damage.
So this works well, and I'm gonna let
the skeleton hit me a couple of times,
and you can see it becomes amber,
and then it becomes red and blinking
when there's only one health left.
Okay?
And that's the end result that we're
looking for because otherwise this
game is just an endless, battle
of just smacking each other silly.
so it's now completed the
implementation of the health bar
and it's running some checks.
So let's do our own check within the game.

So you can see I've loaded up the game
here, and indeed, the health bar is
showing up above the skeleton, and the
character also suffers damage as well.
you can see this is very close to the
end result that we're looking for.
There's just a slight difference
in how the, health looks,
but everything looks correct.
it has gone ahead to correctly play the
death animation, which is interesting
because in my earlier implementation
of the game, it didn't, automatically
implement the death animation for me.
in some cases You will need to prompt
the, AI to implement the death animation.
But in some cases, because it can
check things out from the index.json,
it will automatically implement
the death animation for you.
But now what we, what we want to do
is that we want to say, "On death of
the enemy, the enemy should stay on
the ground for a while before blinking
and disap-disappearing after a bit.
If the player has death, a game
over screen appears allowing
restart or back to menu.
The game should stop movement animations
of any enemies in this case." So that's
what we're gonna do now, and we're
just gonna have it such that the game
just doesn't reach this strange stage
where I can still move my character
around and skeleton standing there.
There's no sense, that the
game is making any progress.
what you'll find is that it's really
fun getting, these new features
in and getting animations in and
getting things on the screen.
But don't forget to close
the loop on your game.
you need to have a game over condition,

and in some cases, you also need
a clear win condition as well.
So we're gonna let the skeleton
hit us a couple of times, okay?
And hopefully, we get a game over screen.
So not only a progression, but you
need to have challenge as well.
There you go.
Game over, So now there's
a progression per round.
We want to have an overall sense of
progression as we go on with the game.
What we're gonna do now is, if you
remember, in this game, is that
each time we defeat the enemies, we
clear that round, and then we get a
next round where more enemies spawn.
Okay?
And the way that you want to do
this is that you want to have
waves of enemies, but you also
don't want this to be too random.
So let me just show you
how to go about doing it.
So what we're gonna do here is that
We're gonna use this following prompt.
We need to have a game loop
with some sense of progression.
We should only have a bunch of rounds,
and we should progressively increase
the number of spawned enemies per round.
only after all enemies are defeated
does the round progress, and we
should have this progression set
up as a configurable parameter.
And I'll explain what this means
But effectively, you want to be
able to say, "Okay, round one has
two enemies, round three has five
enemies," and things like that.
And I say here using Fibonacci increments.
don't bother too much about this for
now, but you want to have some kind
of, increment that makes the game

more challenging over time rather than
just doubling or incrementing by one.
So we will keep track of the
enemies killed and time elapsed
too, and this gives a timer and
a sense of greater progression.
We want a progressive fun loop
that keeps the player coming back.
Easy to learn, progressively harder.
So with this prompt, we are going to
be able to build a game that has a
sense of challenge and progression.
on the topic of Fibonacci, this is
what you call the Fibonacci sequence.
I've come across with
whenever I work in games.
Whenever you're trying to have some
sense of progression, rather than
incrementing by one each time sometimes
adding this Fibonacci sequence gives
you, an automatic sense of progression.
So you have one enemy in first round,
then you have two enemies, then you have
three, then you have five, then you have
eight, then you have 13, then you have 21.
So it becomes tougher as it goes on.
It's not the best, but it's
useful, whenever you need a
nonlinear sense of progression.
if you don't know about a Fibonacci, the
way it works is that you do zero one.
Zero plus one is one, one plus
one is two, one plus two is three,
two plus three is five, and that's
how the sequence comes about.
but don't bother too much about this.
this is, gonna be a big structural change
because this is the core game loop,
and this is gonna take a while longer.
Once you reach this point, you're going to
be very close to having a complete game.
so now let's take a look at whether or
not the, progression system is running.
So we're gonna click on Sandbox.

Oh, you can see round one has begun, and
you can see that the, information around
the round is in the top left-hand corner.
And let's just defeat this enemy
and then... round one is cleared.
Now round two has begun, and now
there are two enemies, so it's gonna
be a lot tougher now, and I'm at
least able to evade the skeleton.
And let's try this.
So you can see there's two, so
the next round, you're gonna have
three, and it's gonna get more and
more difficult as things go on.
But here you are,
You basically have a complete
game right now, right?
You have a sense of progression, you have
a challenge, you have enemies, you have
integrated artwork, you have a timer.
This is looking really good.
So if you've managed to reach to
this stage, give yourself a pat on
the back because this is further
than most people would have gotten.
because they usually reach a stage where,
they have a couple of things running
on screen, but there's no closed loop.
and there's just no sense
that this is a complete game.
closing the loop is the most important
thing when you're building a game, now
once you've reached to this stage, you're
gonna hit a problem, and if you click on
Restart right now, you will see that it
doesn't clean up the game properly, okay?
So you can get stuck here.
Your health is down.
None of the enemies move, and you're
just in this really weird state, okay?
so what you wanna do is
fix the bug up right now.
So you can just say, "There is a bug.
Pressing Restart does

not restart the state.
No enemies are spawned.
Player health is zero." in fact,
in this case, enemies from previous
session are left behind, okay?
So now we're gonna press OK, and
what we're gonna do is press Steer
This time, because we want it to
continue off where it left off.
So it fixed all of these things
related to, having a progression.
But if your restart is not
working, then that is a bug.
Okay?
So steer is like a way for it to continue
from where it was at to then, implement
the fix that you're talking about.
Whereas previously, if you
remember, I pressed Escape,
That was just to interrupt it, and
that was because I knew for a fact
that I wanted it to stop whatever it
was doing so that it can fix the bug.
So steering your conversation
is just another way for you to
be able to get, to this stage.
This is a very common, issue, whenever
you're working with Phaser scenes.
in 90% of the games that I work
with, the restart always, encounters
issues because there's so much stuff
going on within a game session, that,
restarting, needing it to clean up
things and, put back your health, all
of these things, tend to get missed.
So don't be surprised if you encounter
it, and this is the way that you fix it.
Just tell the AI that it needs to clean
up and restart the state and to clear
all the entities that are on the screen.
So it says that it's found the root cause.
It's going to fix up, what's
going on over here because it's
reusing, some existing state.

So what it's gonna do is that
it's going to do a proper cleanup
whenever the game needs to restart.
And, now the restart bug should be fixed,
So let's just take a look and get myself
beaten up by this skeleton over here.
And there we go.
And now I'm gonna click restart, now
we have things running correctly, This
means that we now have a complete loop,
and we are able to restart the game
when we hit the game over conditions.
Okay?
So let's visit.
Oh, there we go.
And this game's a lot harder
now, but now we can restart and
everything is looking correct.
so now what we have right here
is pretty much a complete game.
But one of the things to note is that,
there's something that's missing, right?
the missing thing here is that
there's no sound and there's no music.
remember when I told you that,
when you're building a game, try
to focus on making it complete.
No game is complete if
there's no sound and music.
let me show you a trick to get all of the
sound and music into your game without
you having to spend too much time looking
and searching around for sound effects.
so for us to be able to leverage
sounds within this game, we're gonna
use something called ElevenLabs.
And what we're gonna do is
install the ElevenLabs skill.
The ElevenLabs skill basically allows
you to generate sound effects, music,
and many other things using AI.
And the skills allows you to
effectively just do everything within
your command line or within Codex.

and the only thing that you
need is an ElevenLabs API key.
So this is my ElevenLabs website.
If you go over to the bottom left-hand
corner and go to Developers and then
go to API Keys, you can actually
create a key over here, and then you
can just call it whatever you want.
All right, I'm gonna call
this, Pirate Beat, um, Up.
and what you wanna do is have sound
effects and music generation, that's
the only thing that you really need.
Everything else you can just leave, alone.
you wanna copy this key, and then
what you wanna do is you want to go
into, this folder create an .env file.
This is called an
environment, variable file.
you want to call it ElevenLabs
API key equals to your key here.
I'm gonna do ElevenLabs API
key, and then I'm going to copy,
the key and pop it like that.
Okay, so this is the main thing,
and this allows you to utilize
your ElevenLabs account and to
generate the sounds that you need.
so that's the API key.
So the next thing that you
wanna do is install the skills.
So just copy this, command
that you have over here.
And what you wanna do is open
up the terminal, within your
project and paste it in here.
Just follow the instructions over here.
If you need to install
their packages, say yes.
Okay.
And what it's gonna say is that it
found eight skills, for ElevenLabs.
What do you need?
We just want music, so press Space,

and sound effects, just press Enter.
it will already work with, Codex and
Cursor and all this stuff, and usually,
it will default to having Claude as well.
So you just press Enter here and then
do installation scope to project, and
then you can do symlink recommended.
And what this means is that rather
than having copies, of the skills
for Claude Code and the others, it
will just link both of them together.
Just click OK and then
proceed with the installation.
if you look into your project You
will see that you have a music
skill and a sound effects skill.
This will also work for Claude
Music and Sound Effects.
if you do dollar sign and then
you do music, you will see that
it's pulled it in already, and you
can do dollar sign sound effects.
So what you want to do now is use
this prompt that we have over here.
And it says, "Based on the vibes of
our game, let's generate BGM," our game
that matches the theme of pirates on
a ship." I guess in this case, it's
on a beach, so we can change that.
we're gonna use the music skill,
also add in sound effects across the
board using the sound effects skill.
So I'm gonna do sound effects.
These should be placed in public
assets BGM and public assets
SFX with assets.json, updated.
actually, this should be
assetsindex.json updated.
For music, have four candidates
switchable within the debug panel.
So this is just to give you some
variation, so you can pick from there
I'm going to kick this off right now.
And this, my friends, is, like a pro tip.

if you need music and sound effects for
your game, this is not perfect, but it
will get you quite far along the way.
what you see here, and I'm gonna turn
on the music for you now everything
that you see in this game, has
been generated using that exact
process using that single prompt.
Okay.
So I'm gonna just mute the
what you saw there was the exact same
prompt that I used for that game, And
it basically just put in the background
music for me and all the sound effects,
and I didn't have to do anything more.
So let's just see what happens, over here.
And you will see that sometimes
it can't find the ElevenLabs
API key, so he says he's gonna
check the repo's local .env path.
that's exactly why we went ahead
and put this in, the .env file.
So remember that there needs to be a
dot in front of it, and then it needs
to be named exactly this, so .env.
this is just a convention, for,
the system that we're using,
for loading environment keys.
And when you do this, remember never
ever to share your keys, because if
this key gets leaked, somebody can use
your account to generate their sounds
and it will deplete your credits.
do not share this.
I'm showing it here because I'm gonna
delete this key once this tutorial is
completed, whatever you have in this .env
file, you should never ever share it.
if you're using Git make sure that your
gitignore has the .env so that this does
not get sent into your repository as well.
All right, so this is done now.
It's gotten all of the four different
tracks, as well as the sound effects

for attack, hurt, death, enemy
spawn, and all the other stuff.
so let's just check it out.
Just make sure to go, into your
settings to make sure mute is off.
and let's just see what turns up.
All right, so there you have it,
That's the music and the sound
effects already integrated.
There's some things that you probably
want to tweak, but this gets you
very far along the way, and it makes
the game feel a lot more complete.
I want to finish off this tutorial by
adding some level of polish if you look
at the game that we have over here,
this version, You can see there's a
little bit of what I like to call juice
that makes the game feel more alive.
So you can see there's, the flashing,
there's the screen shake, and
it makes the game feel juicier.
One of the key things here is that
when there's a contact made between the
player and the skeleton, the game freezes
for a very short period of time, and
that makes the impact seem a lot more.
I'm gonna just continue this conversation
that we have over here, and I'm gonna
say add debug toggles for s- for muting
sound and BGM in the debug panel.
And so I'm just gonna let this continue
and implement that because it doesn't
have anything else to do with the game.
But let's now finally move on to
adding the polish that we want.
you can see it's already added the shadows
underneath the actor, so that's great.
So we don't need that.
But what we want to do is that
we want to say we want to have
a noise-based screen shake.
So we don't want to have
random screen shake.

A noise-based screen shake just
make things feel a lot smoother.
minor on enemy hit, but it
shakes more when there's a death.
We want to have freeze
time for the enemy hits.
less when it's just a regular hit
and more on death, and then a white
flash when the player gets hurt
with a cooldown for invulnerability.
And then I want it to have some particle
effects And that's all you need to do
to make the game, feel a little bit
more polished so here we are, and you
can see that the game has already,
implemented the muting, and that allows
me to, not have this playing while
I'm, talking over the video as well.
Okay?
So now you can see compared to what we
saw earlier, there's just no screen shake,
there's no flashing, so ultimately the
game still feels a little bit flat, and
these little bits of juice will help to
make your game feel a lot more complete.
so the polish has completed.
So you can see there's
just a bit of screen shake.
And when the skeleton actually gets
killed, you can see that the screen shake
goes to become a lot more evident, okay?
And if you actually look very
closely, I'm gonna just make this
a bit bigger, so you can see that
there's particle effects now when we
hit the skeleton, and you saw that.
hit, it feels a lot more impactful, right?
so these are the things that you want
to do in order to polish up your game
just to make the game feel a lot better.
And we can just turn on the music again.
I'm gonna switch it to this moon-
All right.
And so there you have it.

We have completed this entire game from
start to finish, So what started out
as a blue square walking around on a
black screen, we have now come all the
way to this fully integrated game with
sound effects, with music, with particle
effects, and with all the different levels
of polish it has a full progression loop.
It has increasing
difficulty of challenges.
and all of this was done, if you
recall, without writing a single
line of code it was all prompted
within the Codex application.
So I really hope you
enjoyed today's tutorial.
Don't forget to give it a like,
hit that subscribe button, and
also turn on notifications so that
you always be the first to know
when new content like this drops.
Now, one thing that we didn't cover
in this video is how I went about
getting hold of these sprites and
animations in the first place.
If you want to learn about how
all these character animations
that you've seen, that we've used
in the game were created using AI.
You'll want to check out this video
where I walk through my entire process
for generating these animations.
So till next time, I'll see ya.